

FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

# **LayerIt: Towards a Time-Aligned Framework for Composable Music Visualization Layers**

**Fernando Azeredo**

WORKING VERSION



**Mestrado em Engenharia Informática e Computação**

Supervisor: Prof. António Sá Pinto

September 9, 2026

# Abstract

Music information retrieval often requires integrating signal-derived, algorithmic, and symbolic information across different representations. Existing visualization tools typically keep notation separate from physical time, while composites that combine them are assembled manually. This thesis presents LayerIt, a Python-based object-oriented framework for composing independent visual representations into unified figures on a shared performance-time axis. Built on a modular layer-composition model, LayerIt treats audio plots, algorithmic outputs, symbolic scores, and other data sources as independent components that can be flexibly combined and regenerated. Figures are exported as SVG, preserving the score’s MEI structure while keeping added components identifiable. As a validation case, LayerIt is used to visualize beat tracking results alongside aligned score and audio representations, demonstrating how the framework enables qualitative analysis of beat tracking outputs, detection of metrical disagreement, and support for beat annotation workflows. More broadly, the project establishes a reusable, extensible foundation for composable score-aligned MIR visualizations.

**Keywords:** Music Information Retrieval • Visualisation • Audio-to-Score Alignment • Beat Tracking • SVG • Object-Oriented • Python

# Contents

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                        | <b>1</b>  |
| 1.1      | Contextualization . . . . .                                | 1         |
| 1.2      | Motivation . . . . .                                       | 2         |
| 1.3      | Objectives, Research Questions and Contributions . . . . . | 3         |
| 1.4      | Developed Project . . . . .                                | 3         |
| 1.5      | Thesis Structure . . . . .                                 | 4         |
| <b>2</b> | <b>Literature Review</b>                                   | <b>5</b>  |
| 2.1      | Visualisation Types . . . . .                              | 5         |
| 2.1.1    | Sheet Music and Score Images . . . . .                     | 5         |
| 2.1.2    | Symbolic Representations . . . . .                         | 6         |
| 2.1.3    | Audio Representations . . . . .                            | 8         |
| 2.2      | MIR Visualisation Tools . . . . .                          | 13        |
| 2.3      | Audio-to-score alignment . . . . .                         | 13        |
| 2.4      | Summary . . . . .                                          | 15        |
| <b>3</b> | <b>Methodology</b>                                         | <b>17</b> |
| 3.1      | Research Approach . . . . .                                | 17        |
| 3.2      | Iterative Development Process . . . . .                    | 18        |
| 3.3      | Scope Decision . . . . .                                   | 20        |
| 3.4      | Evaluation Criteria . . . . .                              | 20        |
| <b>4</b> | <b>LayerIt</b>                                             |           |
|          | <b>Design and Implementation</b>                           | <b>22</b> |
| 4.1      | The Problem and Our Solution . . . . .                     | 22        |
| 4.2      | LayerIt Overview . . . . .                                 | 22        |
| 4.3      | Design Choices and Rationale . . . . .                     | 23        |
| 4.4      | Generating Visuals . . . . .                               | 23        |
| 4.4.1    | Getting the Warped Score . . . . .                         | 24        |
| 4.4.2    | Getting the MAPS file . . . . .                            | 24        |
| 4.4.3    | Providing BT data . . . . .                                | 25        |
| 4.5      | Layer Alignment Method . . . . .                           | 26        |
| 4.6      | Layers on Layers, within Layers . . . . .                  | 26        |
| 4.6.1    | Visual Level . . . . .                                     | 27        |
| 4.6.2    | SVG Level . . . . .                                        | 27        |
| 4.6.3    | Code Level . . . . .                                       | 28        |
| 4.7      | Summary . . . . .                                          | 31        |

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| <b>5</b> | <b>Validation</b>                                                        | <b>32</b> |
| 5.1      | Validation Approach . . . . .                                            | 32        |
| 5.2      | Reproducing Composite Figure Types (RQ1) . . . . .                       | 33        |
| 5.2.1    | Score, audio, and chroma on a shared axis . . . . .                      | 33        |
| 5.2.2    | Score-aligned BT analysis . . . . .                                      | 33        |
| 5.2.3    | Logits and position displays . . . . .                                   | 34        |
| 5.2.4    | Layered time-based display . . . . .                                     | 35        |
| 5.3      | Workflow Capabilities for Annotation and Evaluation (RQ2, RQ3) . . . . . | 36        |
| 5.3.1    | Evaluation workflow . . . . .                                            | 36        |
| 5.3.2    | Annotation workflow . . . . .                                            | 37        |
| 5.4      | Modular Composition Evidence (RQ2) . . . . .                             | 38        |
| 5.5      | Limitations and Difficult Cases (RQ3) . . . . .                          | 38        |
| 5.5.1    | Expressive performances . . . . .                                        | 38        |
| 5.5.2    | Sparse onset annotation . . . . .                                        | 39        |
| 5.5.3    | Manual editing of the final SVG . . . . .                                | 40        |
| 5.6      | Summary . . . . .                                                        | 41        |
| <b>6</b> | <b>Conclusions and Future Work</b>                                       | <b>42</b> |
| 6.1      | Achievements . . . . .                                                   | 42        |
| 6.2      | Future Work . . . . .                                                    | 43        |
| 6.3      | Final Remarks . . . . .                                                  | 44        |
|          | <b>References</b>                                                        | <b>45</b> |

# List of Figures

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                               |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Example of an audio waveform. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                         | 8  |
| 2.2 | Example of an audio spectrum. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                         | 9  |
| 2.3 | Example of Linear spectrogram. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                        | 10 |
| 2.4 | Example of Logarithmic spectrogram. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                   | 10 |
| 2.5 | Example of Mel spectrogram. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                           | 11 |
| 2.6 | Example of a CQT. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                     | 12 |
| 2.7 | Example of a chromagram. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                              | 12 |
| 4.1 | Composition of the LayerIt visualisation system. Each coloured box represents a distinct visual element that composes a higher-level visualisation. . . . .                                                                                                                                                                                                                                                                                                   | 27 |
| 5.1 | LayerIt composition for the opening six bars of Debussy’s <i>Clair de Lune</i> , performed by Maria João Pires. From top to bottom: time-warped score, waveform, mel spectrogram, and chromagram. Cyan dotted lines mark score aligned onsets. . . . .                                                                                                                                                                                                        | 33 |
| 5.2 | LayerIt score-aligned BT display for BWV 856, performed by András Schiff. From top to bottom: time-warped score, mel spectrogram, and normalized waveform with BeatThis beat and downbeat markers. All panels share the performance-time axis. . . . .                                                                                                                                                                                                        | 34 |
| 5.3 | LayerIt activation-and-position display for BWV 856. The upper panel shows BeatThis beat logits and selected beat estimates; the time-warped score occupies the middle panel; and the lower panel shows downbeat logits and estimated downbeat positions. . . . .                                                                                                                                                                                             | 35 |
| 5.4 | LayerIt waveform display an excerpt for BWV 856 by András Schiff, with BeatThis beat positions aligned to the time-warped score. . . . .                                                                                                                                                                                                                                                                                                                      | 35 |
| 5.5 | Comparison of sparse and dense score–performance alignment for an excerpt of BWV 865 performed by András Schiff. From top to bottom: score warped using sparse onsets, waveform with BeatThis beat estimates in red and retained sparse onsets as black dotted lines, score warped using all onsets, and the same waveform with the complete onset set. The sparse MAPS file retains the first and last onsets and every fiftieth onset between them. . . . . | 40 |

# List of Tables

|     |                                                                                                              |    |
|-----|--------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Summary of development iterations . . . . .                                                                  | 19 |
| 4.1 | Required files for LayerIt . . . . .                                                                         | 24 |
| 4.2 | Required .npz fields . . . . .                                                                               | 25 |
| 5.1 | Composite figure types reproduced with LayerIt. . . . .                                                      | 36 |
| 5.2 | Workflow characteristics relevant to the score-aligned visualisation task considered in this thesis. . . . . | 37 |

# List of Acronyms

|             |                                      |
|-------------|--------------------------------------|
| <b>BA</b>   | Beat Annotation                      |
| <b>BT</b>   | Beat Tracking                        |
| <b>DTW</b>  | Dynamic Time Warping                 |
| <b>GUI</b>  | Graphical User Interface             |
| <b>HTML</b> | HyperText Markup Language            |
| <b>JS</b>   | JavaScript                           |
| <b>MIDI</b> | Musical Instrument Digital Interface |
| <b>MIR</b>  | Music Information Retrieval          |
| <b>ML</b>   | Machine Learning                     |
| <b>OO</b>   | Object-Oriented                      |
| <b>OOP</b>  | Object-Oriented Programming          |
| <b>PDF</b>  | Portable Document Format             |
| <b>PNG</b>  | Portable Network Graphics            |
| <b>SVG</b>  | Scalable Vector Graphics             |
| <b>XML</b>  | Extensible Markup Language           |

# Chapter 1

## Introduction

### 1.1 Contextualization

Music Information Retrieval (MIR) is a research field that develops computational methods to analyze, interpret, and retrieve musical information from performance recordings and symbolic representations. MIR bridges a wide range of disciplines, including musicology, signal processing, machine learning, and human-computer interaction. Because MIR is a data-intensive field, visualization plays a crucial role in its research. The way information is selected and represented shapes the conclusions we draw and guides further progress.

The challenge of visualization in MIR is particularly significant because music is an inherently abstract and subjective medium. Translating music into the visual domain is a problem that spans millennia; throughout history, multiple systems of musical notation have been developed in attempts to capture sound in written form, yet this remains an evolving challenge. For MIR purposes, this means that musical data comes in a variety of forms, including audio recordings, symbolic scores, and annotations of expressive qualities. Each of these requires a different visual approach. An audio visualization such as a waveform or spectrogram reveals how a performance sounds, while a musical score conveys how a piece should be performed; both serve distinct and important purposes for different research tasks.



## 1.2 Motivation

MIR researchers often need to inspect algorithmic outputs alongside audio representations and symbolic scores in a temporally coherent manner. This kind of multimodal, time-aligned visualization is central to both the evaluation of MIR algorithms and the development of annotation workflows. Audio-to-score alignment, the task of synchronizing a performance recording with its corresponding symbolic score, is a key enabler of this type of visualization, as it provides the temporal anchor needed to align heterogeneous data sources into a unified view.

This thesis was motivated by two closely related challenges in beat-tracking (BT) and beat-annotation (BA) research. Contemporary BT systems are predominantly based on deep learning. Consequently, the field relies heavily on tools that support annotation and evaluation tasks, both of which are essential for advancing BT research.

Annotation tasks, particularly beat annotation, remain laborious and time-consuming. Existing graphical user interface (GUI) tools such as Sonic Visualiser leave a clear gap in this area: integrating recent state-of-the-art BT algorithms is difficult, and BA-specific features have seen little meaningful innovation. This disconnect between annotation workflows and the algorithms they are intended to support is a concrete obstacle to progress in both domains.

GUI tools present a similar limitation for evaluation: they often lack methods for inspecting algorithmic behaviour in depth. Better tools for evaluating and comparing models with one another are essential for improving them.

Although solutions like Sonic Visualiser [2] are powerful and widely used, their development cannot keep pace with the needs of modern MIR research. Integrating new algorithms or producing specialized visualizations is often impractical within them. A particularly underserved case is the visualization of score-aligned data: combining audio and data representations with a musical score typically involves manual image manipulation, and no unified methodology is widely used in the community.

For these reasons, researchers frequently resort to *ad hoc* solutions that lack reproducibility and are difficult to share or extend. This fragmentation creates a significant obstacle to the collaboration on which MIR research depends. By contributing a flexible, extensible, and community-oriented visualization tool built on established libraries, this thesis aims not only to address an immediate practical gap but also to establish a foundation for a framework that can support the growing complexity and interdisciplinarity of MIR research.

All of the concerns described in this section provide a basis for the objectives and research questions presented in the following section.

## 1.3 Objectives, Research Questions and Contributions

This thesis is guided by three objectives, which in turn lead to three research questions and a set of concrete contributions.

- **Objective 1:** Develop a reusable Python framework for composing score-aligned MIR visualizations based on an OO layer-composition model.
- **Objective 2:** Demonstrate that the framework can generate and regenerate score-aligned visualization compositions from modular layers.
- **Objective 3:** Demonstrate how score-aligned visualizations can provide qualitative visual context for beat annotation and BT evaluation workflows.
- **RQ1:** How can heterogeneous MIR visualizations be combined into a common score-aligned representation?
- **RQ2:** How does LayerIt support the creation, modification, and regeneration of score-aligned visualizations through modular layers and scriptable composition?
- **RQ3:** How can score-aligned visualizations provide qualitative context for interpreting BT outputs and inspecting beat annotation data, and what alignment-related limitations affect this use?

The main contributions of this thesis are as follows:

- The LayerIt framework for composing score-aligned MIR visualizations through an OO layer-composition model.
- The release of LayerIt as an open-source Python framework.
- The open-source repository containing examples of the visualizations developed as part of this work.
- A Late-Breaking Demo paper submitted to ISMIR 2026.

The primary contribution is the layer-composition framework and the figures it enables, rather than the alignment method itself, which is provided by ScoreWarp [8].

## 1.4 Developed Project

To address these objectives and research questions, we developed LayerIt, a Python-based object-oriented (OO) framework for composing score-aligned MIR visualizations. LayerIt integrates established MIR libraries and tools into a Scalable Vector Graphics (SVG)-based workflow in which visual elements are treated as independent layers that can be composed, reordered, and combined into custom figures.

At its core, LayerIt supports the integration of symbolic scores with audio and algorithmic data in a single view. By relying on ScoreWarp for temporal alignment, it makes it possible to overlay spectrograms, onset information, and beat-tracking outputs on score representations within the same visualization. The result is a flexible layer-composition model that supports the visualization workflows developed in this thesis and provides a foundation for future MIR visualization workflows.

## 1.5 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2** provides a literature review covering the main audio visualization types, existing GUI and code-based visualization tools, and audio-to-score alignment.
- **Chapter 3** presents the adopted methodology and describes the five iterative phases that culminated in the development of LayerIt.
- **Chapter 4** describes the problem addressed by LayerIt, its design and implementation, and the “layers within layers” philosophy that underpins its architecture and use.
- **Chapter 5** evaluates the framework against the research questions through the generated figures, a qualitative workflow comparison, and a discussion of its limitations.
- **Chapter 6** reflects on the achievements of the thesis and outlines directions for future work.

## Chapter 2

# Literature Review

In this chapter, we present the background required for this thesis. We begin by exploring the landscape of music visualisation methods, categorising them into three main types: sheet music representations, symbolic machine-readable formats, and audio-based visualisations. This foundation is essential for understanding how different types of musical information can be represented visually and interacted with computationally. We then examine contemporary MIR visualisation tools, distinguishing between GUI-based solutions and code-based workflows to highlight the gap that motivates our work. Finally, we introduce audio-to-score alignment and explore relevant tools and approaches that have informed our solution.

### 2.1 Visualisation Types

We can sum up all music related visualisations into three categories:

1. Sheet Music and Score Images
2. Symbolic Representations
3. Audio Representations

#### 2.1.1 Sheet Music and Score Images

Sheet music describes musical works using a formal language based on musical symbols and letters. Thus, this kind of musical representation always requires a level of musical literacy from the performer. Furthermore, sheet music usually does not convey explicit information about how a given piece should be performed; it is intended to allow the musician a certain degree of interpretative freedom. This can include variations in tempo, articulation, and dynamics, among others.

Sheet music also comes in a variety of forms that vary throughout history and across cultures. For our purposes, we focus on the Western standard of musical notation. Still, within Western music, there can be vastly different forms of notation; however, most are based on a staff consisting

of five horizontal lines and four spaces, each representing a different musical pitch. Musically relevant symbols are placed on the staff to denote pitch, temporal, and expressive indications. [14]

### 2.1.2 Symbolic Representations

Symbolic Representations are machine-readable formats where musical events and expressions are explicitly encoded. These come in various formats, such as MIDI and MusicXML.

Visually we can identify two main applications for this category, these include: piano-roll representations and digital score representations.

Historically piano-roll representations come from so-called “player pianos”, these were self playing pianos that became quite popular in the late 19th to early 20th century. This was achieved by a mechanism that would hit keys of the piano through a continuous roll of paper with perforations. The roll would move over a reading system (known as a tracker bar) that would trigger the musical actions accordingly.

This concept of a continuous and explicit representation of music performance would later prove useful for digital applications. Such is the case for the Piano-Roll.

Today the **Piano-Roll** is a staple feature in any DAW as a visual and interactive tool to write and edit musical events in the form of Musical Instrument Digital Interface (MIDI) notes and attributes such as velocity and other expressive controls.

**MusicXML** refers to the XML-based language used to transcribe musical scores into the digital domain. There are multiple sub-variants of the MusicXML format for specific applications and there are other formats for digital scores, but MusicXML can be considered a “universal” format accepted by most digital score editors.

MusicXML was developed to serve as a universal format for storing and sharing music scores across different music notation applications. Digital music-notation editors then employ proprietary variants of this format to address each program’s specific characteristics. In some cases, these may not even be XML-based, but the MusicXML format can be considered a universal format that is generally accepted and exportable across virtually all such applications. These include, among others:

- **.mscz and .mscx**: These are MuseScore-specific formats named for “MuseScore Format” and “Uncompressed MuseScore”, respectively. Both are XML-based.
- **.sib** - “Sibelius”, a proprietary format of Avid (also non-XML-based)
- **.mei** - The MEI format, XML based.

The .mei format is the one that is essential for this thesis and the most useful for MIR purposes. MEI stands for both the XML-based score format in itself (.mei), and the community that develops it (Music Encoding Initiative). Citing from the MEI website’s introduction:

“The Music Encoding Initiative (MEI) is a community-driven, open-source effort to define a system for encoding musical documents in a machine-readable structure. MEI brings together

specialists from various music research communities, (...) to define best practices for representing a broad range of musical documents and structures.”[1]

It is important to note that MEI focuses on a “machine-readable structure”. This structure relies on unique identifiers (@xml:id) that make every element in the score individually addressable, from individual noteheads to complex spanning elements like slurs and ties. However, more than a music score format, MEI also serves to keep various notation, performance, and interpretation data not only stored but also linked to individual score elements. This is why the MEI format and its community are so essential to MIR as a whole.

That being said, MEI is a purely data-driven project, and thus, in order for it to be visualised or interacted with, it needs to be decoded into a “human-readable format”. That is where Verovio comes in. The MEI and Verovio projects are closely interlinked for that reason. While MEI defines how musical information is stored, Verovio is the primary tool used to transform that data into interactive visualisations and performance-ready outputs.

Verovio is a comprehensive open-source library designed to serve as a native typesetting engine for the MEI format. Verovio’s primary role is to engrave MEI data directly into SVG without losing rich metadata through intermediate format conversions. The usage of the SVG format is an essential one for multiple reasons, quoting from Verovio’s reference book:

“In a web environment, the addressability can be used for highlighting graphical elements such as notes or any other music symbols. One additional characteristic of SVG is that its XML tree can be structured as desired, and an innovative design feature of Verovio was to go further in the structuring of the output by leveraging this characteristic. Since Verovio implements the MEI structure internally, this key feature of SVG made it possible to preserve the MEI structure in the design of the SVG output in Verovio. Preserving the MEI structure in the SVG output is a considerable overhead in the rendering process but makes it a unique feature of Verovio.

As a result, Verovio output in SVG is not the end of a unidirectional rendering process. Quite on the contrary, it should instead be seen as an intermediate layer standing between the MEI encoding and its rendering that can act as the cornerstone for a bi-directional interaction: from the encoding to the notation, but also from the notation to the encoding through the user interface.” [18]

To further demonstrate the usefulness of MEI and Verovio, we next present a quotation from the article about Piano Precision, a GUI software tool for audio-to-score visualisation. We explore this tool further below; for now, the importance of this quotation lies in its demonstration of a practical application of MEI and Verovio.

“Technically, what makes MEI especially useful is that it can be rendered to graphical SVG output while retaining its semantically meaningful XML hierarchy and unique identifiers for every score element (e.g., ties, notes). Piano Precision renders the scores into SVG output using Verovio, an open-source toolkit for visualising

MEI data. This makes it easy to locate interesting objects in a score and, e.g., highlight them in the rendering. The result is interactive images in which users can flip pages, zoom in and out of a score, and click on elements in a score to jump to their playback positions. ” [10]

### 2.1.3 Audio Representations

Audio representations are graphs and/or plots that are intended to represent sound. They are not strictly for representing music; their purpose is to translate sound from the audible to the visual domain. Audio is a term that refers to an acoustic sound that is turned into an electrical signal. Most audio representations are therefore digitally generated by applying mathematical operations to the audio signal, such as the Fourier transform.

We can identify at least three components that define a given audio signal: time, frequency, and amplitude or energy. An audio visualisation will present explicit information about at least two of these.

Waveforms are among the most common and simplest ways to represent audio. A waveform consists of a single line in a two-dimensional plot, with amplitude on the y-axis and time on the x-axis. This representation does not provide frequency information; it only shows how the amplitude of the audio changes over time.

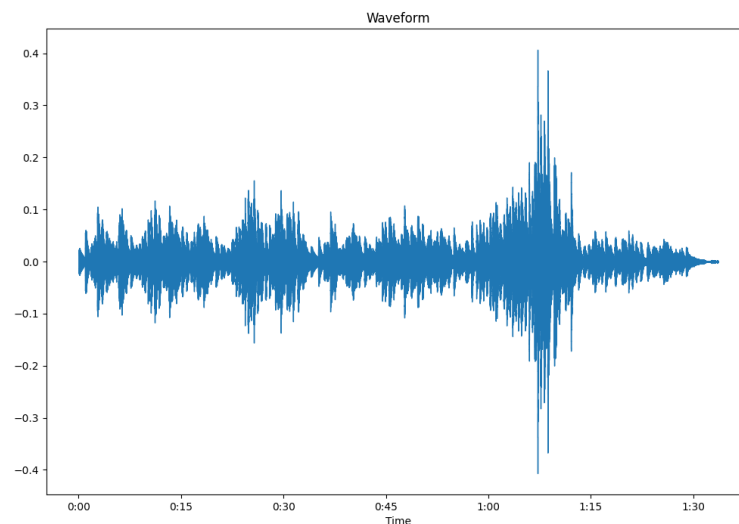


Figure 2.1: Example of an audio waveform.

A spectral visualisation, or a spectrum, refers to a graph with only frequency and amplitude/energy representation in a given time window.

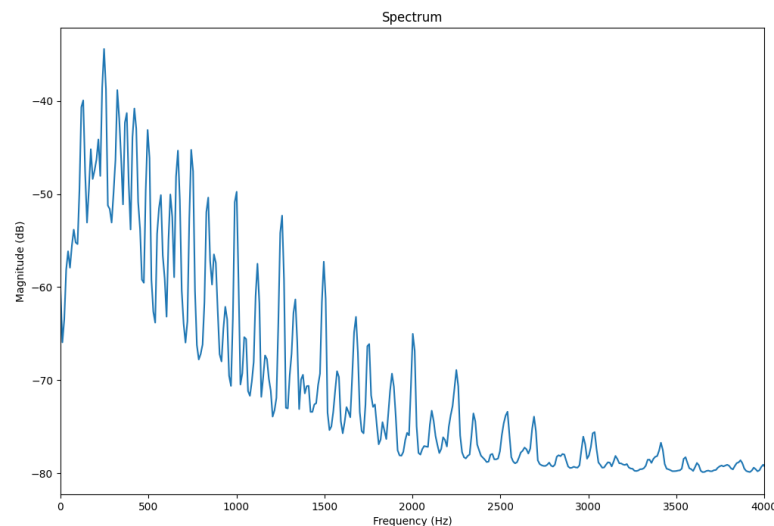


Figure 2.2: Example of an audio spectrum.

To display the three sound components, the plot needs to represent three distinct dimensions. The spectrogram achieves this by placing frequency on the y-axis, time on the x-axis, and colour in place of the third axis (amplitude), thereby displaying all three parameters in a single two-dimensional visual plot. There are multiple variants of the spectrogram, each with different applications.

The way frequencies are displayed in a given spectrogram can help us understand audio in particular situations. There is a wide variety of spectrogram representations that can vary in how their contents are displayed.

The linear spectrogram is the most basic form of spectrogram, in which frequencies are displayed on a linear scale. This means that frequencies are equally spaced on the y-axis. For musical applications, this kind of representation tends to be less useful, as it does not reflect the way humans perceive sound.



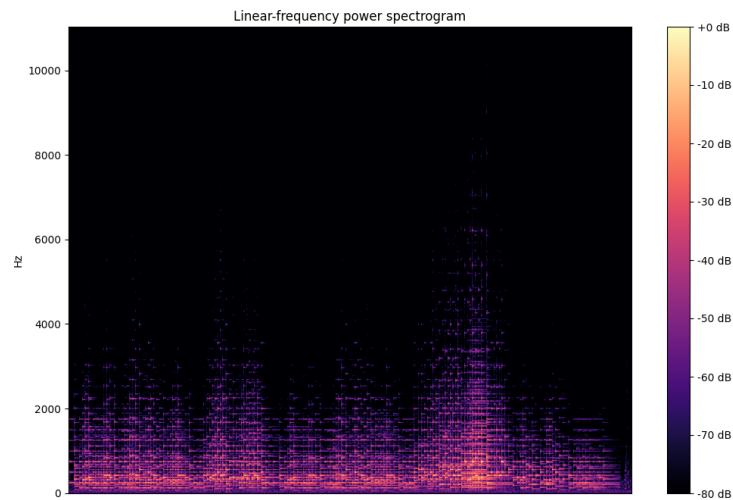


Figure 2.3: Example of Linear spectrogram.

The logarithmic spectrogram displays frequencies in a logarithmic scale. This mimics better the way humans perceive sound.

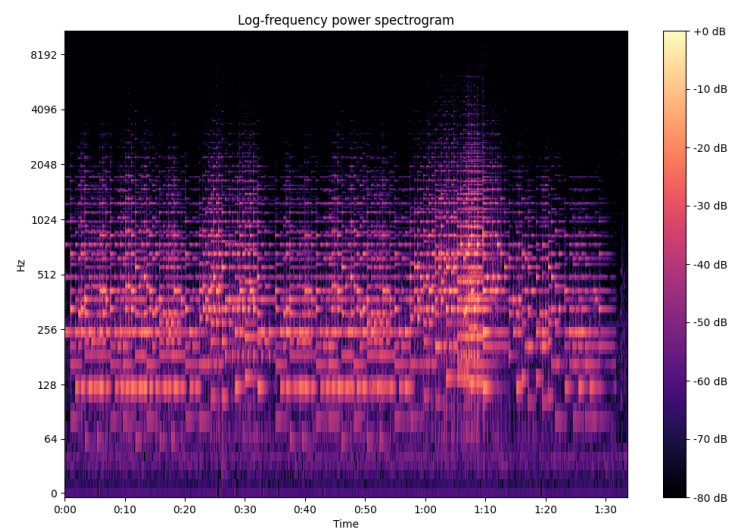


Figure 2.4: Example of Logarithmic spectrogram.

The Mel spectrogram also displays frequencies while taking the human hearing curve into account. The difference is that the logarithmic spectrogram simply organises frequencies logarithmically, whereas the Mel spectrogram organises frequencies according to the Mel scale. This is a perceptual scale of pitches judged by listeners to be equal in distance from one another. For music-related applications, this type of spectrogram is more useful, since it allows us to distinguish more clearly between musical pitches.

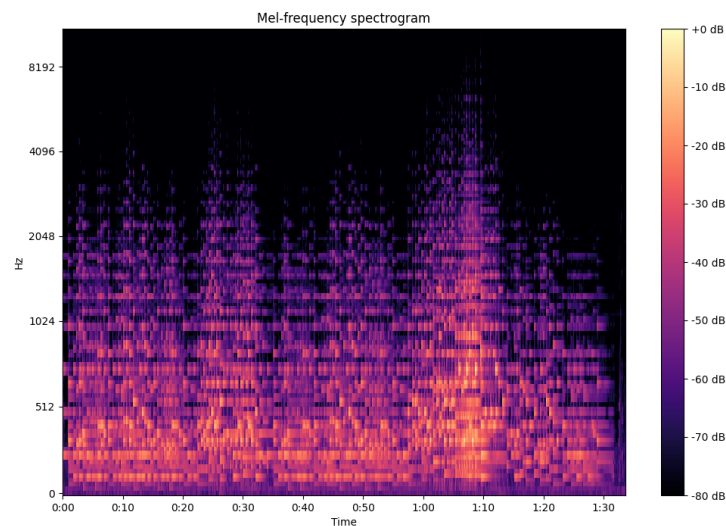


Figure 2.5: Example of Mel spectrogram.

The Constant-Q Transform (CQT) and chromagram are visual representations that derive from the spectrogram. Whereas a spectrogram shows frequency data, the CQT and chromagram can be used to visualise note-related information: tone height in the case of the CQT and chroma in the case of the chromagram. The following quote explains these terms:

“(...) a pitch can be separated into two components, which are referred to as tone height and chroma. The tone height refers to the octave number and the chroma to the respective pitch spelling attribute contained in the set {C, C♯, D, (...), B}.” [14]

The CQT organises its frequency content using geometrically spaced frequency bins. This is done to resemble the equal-tempered system.

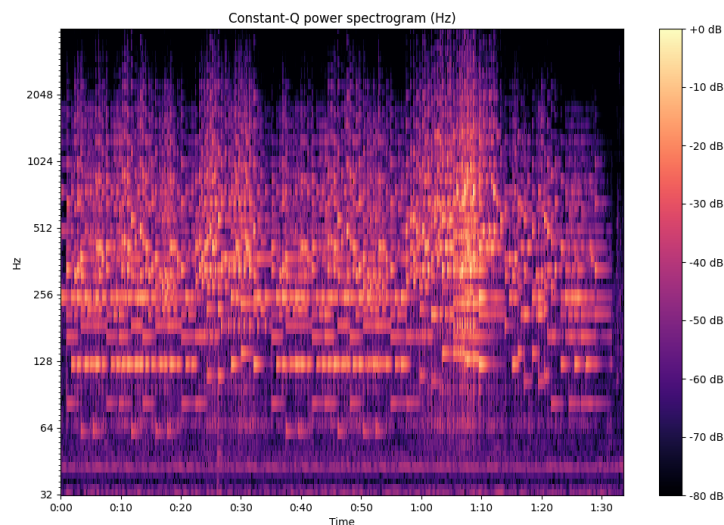


Figure 2.6: Example of a CQT.

While the CQT can distil the frequency spectrum into bins that correlate with individual notes, the chromagram takes this a step further by visually summing the energy of each note across all octaves. Thus, in the end, we obtain a plot divided into 12 lines, each representing a given note. In its essence, this type of visualisation shows the use of the same notes across the recording. A chromagram can be useful for determining harmonic and scale-related information.

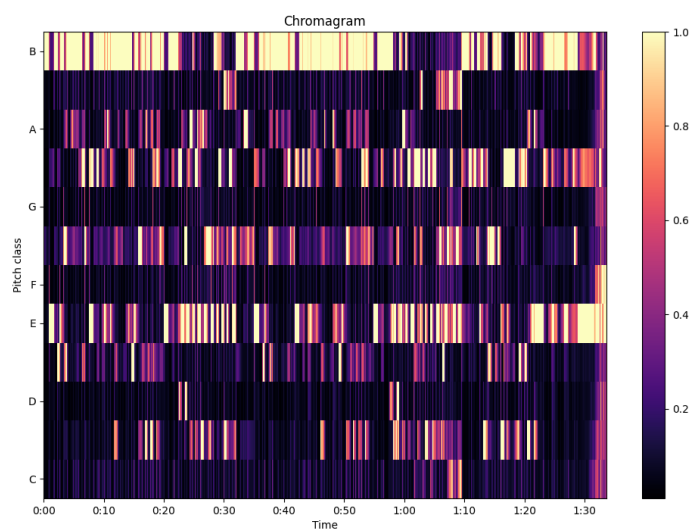


Figure 2.7: Example of a chromagram.

[11, 12, 14]

## 2.2 MIR Visualisation Tools

Within MIR, visualisation workflows typically fall into two broad categories: GUI-based tools and programming-based workflows. For this thesis, the most relevant comparison is between Sonic Visualiser as a representative GUI tool and code-based workflows used for custom MIR analysis.

Sonic Visualiser [2] is a dedicated program for audio visualisation and analysis, and it is one of the most common tools in MIR practice. It is well-suited to inspection, annotation, and exploratory analysis, making it a strong baseline for research-oriented visual work. A particularly relevant example is Piano Precision, a Sonic Visualiser-based GUI tool for visualising score-aligned spectrograms.

“The Piano Precision UI is adapted from Sonic Visualizer, with an additional area displaying a score (...) The interface is designed to help streamline the processes of loading a digital score and a performance recording, editing and visualising annotations (...), and exporting score-aware annotations.”[10]

While modern GUI-based tools such as Sonic Visualiser are powerful, they do not always provide the flexibility required for state-of-the-art MIR research. Many research questions demand highly specialised visualisations that are not available in pre-built software, especially when multiple data types such as audio, scores, annotations, and computational outputs must be combined in a single figure. In these cases, programming-based methods are often necessary.

Code-based workflows are also attractive because they can be versioned, shared, and included directly in publications, which improves reproducibility. This is especially important in academic research, where results must be inspectable and repeatable. In addition, MIR increasingly overlaps with machine learning and large-scale data analysis, which naturally favours scripted workflows for feature extraction, training, evaluation, and custom visualisation.

Currently, Python is the go-to language for most of MIR research. Libraries such as Librosa and Matplotlib, together with tools such as Jupyter Notebooks, are commonly used to implement these workflows.

Taken together, these reasons explain why code-based visualisation remains essential in MIR and why Piano Precision is a useful motivating example for this thesis: it sits precisely at the intersection of interactive GUI support, audio-to-score alignment, and reproducible computational workflows.

## 2.3 Audio-to-score alignment

Audio-to-score alignment, also referred to as score matching or score alignment, consists of synchronizing an audio recording of a musical performance with its corresponding symbolic score [3]. In practical terms, the goal is to establish correspondences between what is heard in the recording and what is represented in the score, enabling tasks such as score following, guided listening, interactive navigation between audio and score, and research-oriented analysis.

Time To Align! distinguishes graphical, logical, and physical temporal domains and provides a general model for recording alignments among them [9]. In this model, an alignment is a reusable artefact that records correspondences and their provenance across timelines. LayerIt addresses a complementary problem: it uses an available score-performance alignment to compose graphical, symbolic, and audio-derived representations into a shared score-aligned visualisation.

The purpose of an alignment is not limited to algorithmic matching. Thomas et al. [21] describe score–audio linking structures as a basis for accessing and exploring multimodal music sources within a single framework. LayerIt uses an available linking structure for a more specific purpose: the generation of static, score-aligned visualisations. It does not compute the alignment itself. Existing synchronisation toolkits, such as Sync Toolbox, provide dynamic-time-warping-based pipelines that could supply an upstream alignment in future work [15].

In the literature, it is useful to distinguish between two broad alignment strategies. The first is **time-warped alignment**, in which the score is mapped onto a continuous performance timeline. This approach preserves a direct temporal relationship between score elements and audio playback, and it is especially relevant when every relevant musical event needs to be placed along the time axis. The second is **link-based alignment**, in which the relationship between score and performance is established through discrete anchor correspondences. Rather than continuously warping the score, this approach links selected score positions to performance positions and uses those reference points to organize the alignment.

These two strategies are not separate tasks so much as different ways of representing the same underlying correspondence problem, and the distinction becomes important for the rest of this thesis. Time-warped alignment is the more continuous formulation, while link-based alignment is the more anchor-driven formulation. This terminology will be used throughout the following chapters.

Alignment systems can also be classified by when the matching is performed. **Offline alignment** processes a complete recording after the performance has finished. Because it can access the full signal, offline alignment can use future information to determine the best possible correspondence, which makes it suitable for analysis, annotation, and audio editing. **Online alignment**, also known as score following, performs the matching in real time while the audio is being received. Since it cannot rely on future information in the same way, online alignment typically has to trade off latency against precision.

Beyond this temporal distinction, alignments may be carried out at different levels of granularity. The three most common are **note-level**, **beat-level**, and **segment-level** alignment.

**Note-level** alignments are score matching processes that aim to synchronise at the individual note level. At its core, note-level alignment is based on note and onset/offset detection. In MIR literature it can also be referred to as “fine-grained alignment” because of the precision it requires. Contemporary audio-to-score alignment research focuses on this type of alignment not only because of its precision but also because it can retrieve data relevant to other MIR fields and tasks. However, pure note-level alignments might falter if the performance deviates from the score, for example if the performer skips, repeats, or adds notes or score sections. Repeats and jumps are

a well-established source of difficulty in score-performance synchronisation [7]. For this reason, some score-matching solutions combine fine-grained alignment with other methods.

Piano Precision is an example of an application of a link-based note-level score-alignment method. While the recording is playing Piano Precision shows a spectrogram and waveform that follow along a conventional playback cursor while, on a side window that contains the score, highlighting the corresponding elements being played in the score, note by note. This enables annotations to be mapped to both the audio and individual score elements simultaneously. The authors note the usefulness of this tool for performance researchers [10] however, we will make the case that such feature is useful for BT and MIR research more broadly.

**Beat-Level** alignments aim to follow along beat and/or measures dictated in the score, and then linking them to the performance. Early score aligners [4] would rely on such methods. Today, score alignment solutions, although mostly are note-level types, still combine Beat-Level alignment layers within their algorithms. A good example of this comes from [17], in it the authors improved on other alignment models, one strategy employed was adding analysis of “alignments at both the beat and the bar level”. The beat level alignment would serve as “anchor points” in order to prevent “cascading errors, as misalignment will not propagate beyond the next anchor point”, they noted that this improved results of automatic alignments.

**Segment** type alignments, map broad portions of performance audio to corresponding visual blocks in the score. This type of score matching is mostly made manually, it is common to see it in follow-along videos to help viewers listen to the music while visualising the score. This type of alignment is considered “weak but musically meaningful” [11] since segments can be deliberately chosen i.e., to better show melodic or harmonic structure.

Among the visual alignment tools relevant to this thesis, ScoreWarp is especially important. ScoreWarp presents a score that is tied to a physical time axis, where each musical element is shifted so that its visual position coincides with the time at which it sounds [8]. In other words, it implements a time-warped visualisation of the score while preserving the semantic structure of the original MEI data. This makes ScoreWarp a strong example of how alignment can be used not only as an algorithmic task, but also as a visual framework for interacting with music information. In the next chapter, we will return to this tool in more detail, since it became one of the key references for the development of this thesis.

## 2.4 Summary

This chapter provided a comprehensive overview of MIR visualisation techniques and tools, establishing the foundation for understanding audio-to-score alignment.

We began by categorising music visualisations into three main types: traditional sheet music representations, symbolic machine-readable formats, and audio-based representations. This established the importance of MEI and Verovio, as well as the use of SVG in this context.

We then examined the contemporary landscape of MIR visualisation tools, distinguishing between GUI-based solutions such as Sonic Visualiser and Piano Precision, which provide user interfaces for audio analysis, and code-based workflows that afford researchers greater flexibility to develop specialised visualisations tailored to specific research questions. Python-based approaches using libraries such as Librosa and Matplotlib have become widely used in MIR research, particularly because of their integration with machine-learning pipelines and reproducibility requirements.

Finally, we introduced one of the core tasks of this thesis: audio-to-score alignment. This process synchronizes musical performances with their corresponding symbolic scores. ScoreWarp emerged as a particularly relevant tool, demonstrating how MEI and Verovio can be leveraged to create visually warped scores that maintain semantic integrity while aligning with performance time.

## Chapter 3

# Methodology

This chapter presents the methodological choices that guided the development of this thesis. It explains the overall research approach, the iterative process used to build and refine the work, the scope boundaries that define what the project does and does not cover, and the criteria used to evaluate the end result.

### 3.1 Research Approach

This thesis follows a design-oriented and implementation-driven research approach, supported by iterative prototyping. Rather than treating the problem as a purely theoretical exercise, the work focuses on building and refining a practical framework for time-aligned music visualisation that responds to the needs identified in the literature review and in the reference tools analysed in the previous chapter. The research process is therefore grounded in making, testing, and adjusting concrete visual and interaction decisions as the project evolves.

The methodology combines conceptual analysis with software development. First, the problem space was studied through the comparison of existing tools, visualisation strategies, and alignment workflows. That understanding was then used to define a focused system design that could support layered, time-aligned representations of musical data. From that point onward, implementation and evaluation proceeded together: each prototype was used to verify whether the proposed interaction model, visual structure, and alignment logic were adequate for the thesis goals.

This approach is appropriate because the main contribution of the thesis is not only to describe a concept, but to demonstrate how such a concept can be materialised into a workable framework. As a result, the methodology emphasizes practical feasibility, visual clarity, and alignment consistency, while still remaining informed by the academic context established in the literature review.



## 3.2 Iterative Development Process

The development of LayerIt followed an iterative path made up of five prototype cycles before the final framework took shape. Rather than moving from a fixed specification directly to implementation, the project evolved through successive experiments in which each prototype was built to answer a specific question, evaluated for what it revealed, and then either discarded or used as the basis for the next step. Seen in this way, the value of the iterations is not only in the software they produced, but in the design knowledge they accumulated.

We started by creating **BeatMapJS**. Its goal was to explore a web-based beat-annotation interface and to test whether existing beat-tracking tools could be combined with a more interactive annotation workflow. The prototype was built in JavaScript and HTML using BeatThis [5] and Wavesurfer.js, and it included an interactive spectrogram, region annotation, a tempo-annotation table, and import/export support for SV-compatible text files. The main lesson was that, although the system was functional, it did not contribute a meaningful new annotation capability beyond the integration of BeatThis. More importantly, it did not produce a reusable Python library, which, from the start, was an important requirement. The decision carried forward was that a reusable Python core would be mandatory.

The second cycle was **MEI-Basic-Edit**. Its goal was to test whether manual score alignment could support audio-to-score annotation by allowing score elements to be adjusted against an audio layer. The prototype was built in JavaScript and HTML with Verovio, MEI, and SVG, and it rendered a score below a static spectrogram while allowing users to drag score elements horizontally to align them with the audio representation. It also included an embedded XML editor for direct manipulation of the underlying score code. The key lesson was that interactive score editing was expensive both conceptually and technically: it increased UI complexity and required a level of implementation effort that was not justified by the results. At the same time, this cycle provided valuable familiarity with MEI, Verovio, and SVG structure. The decision carried forward was that SVG would remain a viable composition substrate, but interactive editing would not be the main direction.

The third cycle was **BeatPrecision**, a fork of Piano Precision. Its goal was to extend an existing GUI environment with beat-annotation features while preserving the score-alignment functionality already present in the software. The prototype was built in Qt/C++, integrating BeatThis and implementing horizontal score-alignment alongside the existing spectrogram view. While functional, the project revealed significant challenges. Piano Precision's (and by extension Sonic Visualiser's) codebase proved complex and difficult to manage. Long compile times and the sheer size of the codebase made each developed modification increasingly challenging to implement, manage, and debug. More importantly, this iteration taught us a critical lesson: the need to distinguish between the interaction and visualisation problems in BA. We realized we could not tackle both simultaneously and would need to choose a direction. Recognizing a clear gap in the field, we opted to focus on the visualisation aspect, as we saw greater potential for the thesis to benefit the community in this area.

The fourth cycle was **Score-Alignment.py**. Its goal was to explore score-aligned spectrograms in a Python environment and to assess the Modusa framework as a possible basis for the project. The prototype was built in Python with Modusa and Matplotlib and produced an interactive spectrogram. The lesson was that a truly fluid interactive visualisation, with zooming, panning, a timeline, and playback control, was much harder to implement than expected. Even though the prototype worked, the result was not satisfactory enough to justify continuing in that direction, and the return on effort was not viable. The decision carried forward was to abandon real-time interactivity and instead prioritize static, high-quality, reproducible output.

The fifth cycle was **Score-Alignment.js**. Its goal was to revisit the aligned-spectrogram idea using the lessons learned from the earlier MEI and beat-annotation prototypes. The prototype was built in JavaScript and HTML with Verovio, MEI, and Wavesurfer.js, and it displayed a MEI score alongside a spectrogram while storing beat timestamps in the MEI data and highlighting notes on interaction. The lesson from this final exploratory cycle was that the idea was promising, but it also confirmed the need for a tool that was strictly focused on visualisation, written in Python, and organized using OO principles so that it could be reused and extended more easily. The decision carried forward was that this combination of requirements defined what would then be LayerIt.

Table 3.1: Summary of development iterations

| Iteration          | Goal                                                         | Tools Used                                                                                          | Lesson                                                                       | Conclusion                                                 |
|--------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------|
| BeatMapJS          | Explore web-based beat annotation                            | JS/HTML, BeatThis, Wavesurfer.js, interactive spectrogram, tempo table, SV-compatible import/export | No new annotation feature beyond BeatThis; no reusable Python library        | Reusable Python core is mandatory                          |
| MEI-Basic-Edit     | Test manual score alignment against audio                    | JS/HTML, Verovio, MEI, SVG, drag-to-align score, XML editor                                         | Interactive score editing is costly; gained MEI/Verovio/SVG familiarity      | SVG as composition substrate; no interactive editing focus |
| BeatPrecision      | Extend GUI beat annotation in an existing score-aligned tool | Qt/C++, BeatThis, horizontal score follow-along                                                     | Codebase was hard to extend; interaction and visualisation must be separated | Focus on visualisation, not the interaction problem        |
| Score-Alignment.py | Explore score-aligned spectrograms in Python                 | Python, Modusa, matplotlib, interactive spectrogram                                                 | Fluid interactivity proved impractical                                       | Prefer static, high-quality, reproducible output           |
| Score-Alignment.js | Revisit aligned spectrograms with richer score interaction   | JS/HTML, Verovio, MEI, Wavesurfer.js, beat timestamps in MEI, note highlighting                     | Promising, but confirmed need for a Python OO visualisation tool             | Define the LayerIt conceptual baseline                     |

Taken together, these five cycles show a deliberate narrowing of scope. The project moved from an interactive beat-annotation GUI toward a reusable OO visualisation framework. Each

prototype clarified one part of the problem and ruled out another, so the final system could be built on a more stable understanding of what the thesis needed to solve.

### 3.3 Scope Decision

The development process described above led to an important scope decision. At the outset, the project was framed broadly as a possible BA/BT graphical user interface, with both annotation and visualisation treated as potential contributions. However, the iterative prototypes made it clear that these were, in practice, two different problems, each with its own technical demands, design goals, and implementation costs. Attempting to solve both within a single master's thesis would have diluted the contribution and made it harder to reach a result that was both robust and meaningful.

For that reason, the project deliberately narrowed its focus from an interactive BA/BT GUI to an OO visualisation framework. The interaction problem proved too broad to address properly alongside the visualisation problem, while the visualisation side revealed a clearer and more relevant gap: the lack of a reusable, flexible, and well-structured Python tool for producing layered time-aligned musical data visualisations. By focusing on this gap, the thesis could concentrate on a contribution that was both feasible and valuable.

As a result, this thesis does not attempt to provide a complete BA/BT annotation environment, nor does it seek to solve the broader interaction problem in full. Instead, it focuses on the visualisation dimension, with particular emphasis on clarity, modularity, and alignment consistency. This narrower scope makes it possible to develop a more coherent contribution and to evaluate it against well-defined criteria.

### 3.4 Evaluation Criteria

The evaluation of this thesis is guided by a small set of criteria chosen to reflect the actual purpose of the framework. Since the goal is not to produce a general-purpose MIR platform, but rather a reusable visualisation tool, the evaluation focuses on whether the final system succeeds in covering the visualisation cases that motivated its development, while keeping the implementation manageable and the output reliable. In this sense, the framework is judged less as a standalone demo and more as a deliberate response to a specific research and design problem.

Firstly, the framework should be able to represent the kinds of score-aligned visualisations required by the thesis, including the layered combination of symbolic score data with audio-derived and annotation-derived information. Secondly, it should support modular composition and regeneration: existing visual layers should be combined, modified, and re-rendered through a repeatable procedure rather than manual image alignment. Thirdly, the output should be reproducible. Because the thesis is situated in a research context, the visual outputs should be generated through a process that can be repeated, inspected, and shared without ambiguity.

Two further criteria are extensibility and fidelity. Extensibility refers to whether the framework can be adapted to new visual layers or new combinations of existing ones without requiring major restructuring. Fidelity refers to whether the resulting figures preserve the musical and temporal relationships they are intended to represent, while still making clear the limits of the visual abstraction. In other words, the evaluation must consider not only what the framework can display, but also where its representation becomes less precise, less general, or less appropriate. Defining these criteria here makes the evaluation of the thesis read as a deliberate design assessment rather than as a mere presentation of the implemented tool.

## Chapter 4

# LayerIt

## Design and Implementation

This chapter presents LayerIt <sup>1</sup> as the concrete outcome of the research process described in the previous chapter. Its role is to show how the thesis moves from a problem statement and a set of methodological decisions to a working framework for composing score-aligned MIR visualisations. The chapter is organised from the problem definition to the system overview, then to the design choices that shaped the implementation, and finally to the mechanism used to combine the different visual layers into a single result.

### 4.1 The Problem and Our Solution

The central problem addressed by LayerIt is that score-aligned MIR figures are often created through a mixture of manual editing, *ad hoc* scripting, and tool-specific workarounds. In practice, this makes it difficult to produce figures that are reusable, reproducible, and easy to adapt when the underlying data changes. Existing GUI tools are useful for inspection and annotation, but they are not designed to support the kind of compositional, publication-oriented visual output required by this thesis.

LayerIt responds to this problem by treating visual elements as composable layers rather than as a single fixed image. The framework provides a structured way to generate, combine, and align music-related visual materials in SVG-based output. ScoreWarp provides the alignment backbone. LayerIt’s contribution is an OO layer-composition model that supports the creation, modification, and regeneration of visual compositions.

### 4.2 LayerIt Overview

LayerIt is a Python framework designed to compose score-aligned MIR visualisations from independent visual components. At a high level, it takes aligned musical data, generates the corre-

---

<sup>1</sup>[https://github.com/FernandoAze/LayerIt\\_WIP](https://github.com/FernandoAze/LayerIt_WIP)

sponding visual representations, and arranges them into a layered figure that preserves the temporal relationship between the score and the accompanying audio-derived or annotation-derived information.

The framework is modular: the same base structure can support different combinations of score material, spectrograms, annotations, beat information, and other derived data. These combinations can be modified and regenerated through scripts rather than reconstructed manually in an image editor.

The project uses `librosa` [13] and `matplotlib` [20] to generate audio and data visualisations. The score layer is rendered through `ScoreWarp`. The final output is an SVG file that combines the score with audio and data layers. The BT examples we are going to present were produced with `BeatThis` [5], in order to illustrate how precomputed BT outputs can be incorporated to produce beat-data visualisations.

### 4.3 Design Choices and Rationale

LayerIt uses `matplotlib` to generate audio and data representations when SVG is the final output format. `Verovio`, and subsequently `ScoreWarp`, render the score in SVG. Exporting `matplotlib` output to SVG therefore provides a practical route for combining chart-based visualisations with the rendered score.

SVG is a vector format, so representations that can be expressed as Cartesian plots, such as continuous curves and time-based annotations, can be exported without resolution loss. Other representations, particularly heatmaps such as spectrograms, are ordinarily rendered as raster images. Although SVG can embed such images, it is a static format and does not provide the interactive zooming and panning of a dedicated visualisation interface. LayerIt therefore supports PNG output for raster-based plots. This trade-off preserves the resolution independence of the score and vector plots, while raster layers may lose detail when enlarged and require greater storage.

Because SVG is a text-based markup format, it can also be inspected and modified directly. It can therefore serve as an intermediate representation between codebases written in different languages, such as Python, JavaScript, or C++. This property supports the potential integration of LayerIt into future MIR applications.

LayerIt was developed as a Python-based OOP tool using established MIR libraries, including `librosa` and `matplotlib`. This choice keeps the framework compatible with common Python-based research workflows.

### 4.4 Generating Visuals

LayerIt was developed to generate visualisations for BA and BT tasks, so the current version focuses on these uses. It uses audio-to-score and data alignment to place relevant representations on a common visual timeline. In its current state, the codebase relies on the following files to generate complete visualisations:

| File       | Description                                                                                  |
|------------|----------------------------------------------------------------------------------------------|
| .wav       | File with the audio recording of the performance                                             |
| .mei       | XML Score of the musical piece                                                               |
| .maps.json | Contains data that links onsets with score elements of the .mei                              |
| .svg       | The score in visual format generated by ScoreWarp (requires .mei and .maps <i>a priori</i> ) |
| .npz       | File with algorithmic BT data                                                                |

Table 4.1: Files required by LayerIt to generate complete visualisations.

#### 4.4.1 Getting the Warped Score

The warped score can be generated using the [ScoreWarp online demo](#). The user uploads the score in .mei format together with the MAPS file, then downloads the resulting score in .svg format.

The .mei file contains the score in XML format. A score in .musicxml format can be converted to .mei, for example with the [Verovio online editor](#) or MuseScore. In either case, the original score must be available in .musicxml format before conversion.

#### 4.4.2 Getting the MAPS file

The MAPS file records performance onsets and links them to the corresponding score elements in the .mei file through their `xml:id` attributes. Two methods can be used to obtain this file.

The first method, described in [8], uses the MIDI-to-MIDI matching algorithm of [16]. It requires both the .mei score and a MIDI performance. Examples from the ASAP dataset [17] can facilitate this process. When a MIDI performance is unavailable, the MAPS file must be generated independently.

The MAPS file is a .json file. For the current version of LayerIt, it must provide score and performance onset data in the following form:

Listing 4.1: Minimal structure of the MAPS.json file to work with LayerIt and ScoreWarp

```

1 {
2   "obs_mean_onset": 0.8533333333,
3   "xml_id": ["x1iw1eq1"],
4   "obs_num": 1
5 }
```

In this example:

- `obs_mean_onset` is the time in seconds of a given onset.
- `xml_id` is the `xml:id` of the corresponding onset note in the .mei score. For a chord onset, this attribute can contain more than one `xml:id`.
- `obs_num` is a sequential counter for the supplied onsets.

LayerIt relies on MAPS in two complementary ways: directly for onset visualisation, and indirectly through ScoreWarp, which requires MAPS to generate the warped score aligned with the timeline SVG element which constitutes the alignment reference for all overlaid data.

MAPS is a purpose-specific alignment representation for this workflow. More general interchange formats, such as the match file format, encode note-level correspondences between scores and performances [6]. LayerIt does not currently read or produce match files; supporting such formats would broaden the range of alignment pipelines that can provide its input data.

ScoreWarp performs best when MAPS contains dense onset annotations across the full performance. However, it can still align scores when annotations are sparse via interpolation between available onsets and their `@xml.id` anchor points. In practice, this allows different precision levels (i.e., downbeats, phrase boundaries, or measure-level anchors), trading annotation effort for alignment precision. Even minimal anchors (first and last onset) may be usable for short performances or segments, but intermediate alignment becomes less reliable due to interpolation error.

For the examples provided with LayerIt, Piano Precision was modified to obtain this data. Piano Precision’s Piano Aligner plugin [10] matches performance onsets to score elements defined by the `.mei` file. Piano Precision displays these correspondences in a Sonic Visualiser time-instance layer. The modified layer displays the `xml:id` of each onset rather than its score position.

The time instances can then be exported as `.txt` files containing the minimum required MAPS data using the `TXT_to_Maps()` function in `src/functions/warp_score`.

#### 4.4.3 Providing BT data

Contemporary BT algorithms commonly produce a beat activation function: a continuous curve representing the estimated likelihood of a beat at each frame. Visualising this curve alongside the discrete beat estimates exposes the model output from which those estimates are derived and provides qualitative context for interpreting them.

LayerIt does not run a BT algorithm itself; it consumes a pre-computed `.npz` file containing the algorithmic output. For the examples in this thesis, this file was produced by running `BeatThis` on the performance recording and saving its output with `numpy.savez`. Any other beat tracker can supply this data, provided the `.npz` file follows the structure expected by the beat layers in `src/functions/Beat_Layers.py`. The file should contain five key components:

| Field                            | Purpose                                                    |
|----------------------------------|------------------------------------------------------------|
| <code>beat_times</code>          | Time coordinates (seconds) aligned with probability frames |
| <code>beat_activation</code>     | Continuous beat activation function values                 |
| <code>downbeat_activation</code> | Continuous downbeat activation function values             |
| <code>detected_beats</code>      | Discrete beat positions derived from peak detection        |
| <code>detected_downbeats</code>  | Discrete downbeat positions from peak detection            |

Table 4.2: Fields required in the `.npz` file for BT visualisation.



The activation fields capture continuous model outputs across time, while the detected positions represent the final beat estimates. This separation enables the visualisation of both the continuous activations and the discrete estimates.

## 4.5 Layer Alignment Method

LayerIt currently provides tools for generating visualisations for BT and BA tasks, but the framework is not limited to these applications. Layers can be SVG or PNG elements that span the same, or a proportional, width as a given timeline. Other SVG or PNG plots can therefore be incorporated when they use the same temporal extent.

When the warped score is generated, ScoreWarp can add a visual timeline for the performance. This timeline may span a greater width than the recorded performance and can extend beyond the visible SVG canvas. Because it provides the reference used to position score elements, LayerIt also uses it to align the visual layers with the recording.

LayerIt aligns layers as follows:

1. It reads the SVG containing the warped score, rendered by ScoreWarp, and retrieves the full length of the generated timeline in pixels.
2. It obtains the duration of the audio recording on that timeline.
3. It calculates the recording length in pixels by proportional scaling from the recording duration and the full timeline length.

In the codebase this is done through a single method:

```
Visualizer().get_Layers_WidthHeight()
```

LayerIt does not yet provide conversion methods for arbitrary external visualisations. However, the alignment procedure can be applied to an external image by placing its left edge at the start of the relevant timeline and scaling it to the recording length calculated by the framework. The image can then be scaled to the timeline length of the score generated through ScoreWarp.

## 4.6 Layers on Layers, within Layers

LayerIt combines aligned visualisations of algorithmic data, audio, and symbolic music representations, such as scores, that share a common timeline. Its Python OO architecture represents these visualisations as layers that can be composed into larger outputs. This organisation supports the modification and regeneration of the visual compositions described in Chapter 5.

The visualisations generated by LayerIt follow a “layers within layers” philosophy: visual elements are self-contained components that can be composed into upper-level elements, which in turn can be composed into yet higher levels. [Figure 4.1](#) demonstrates this methodology.

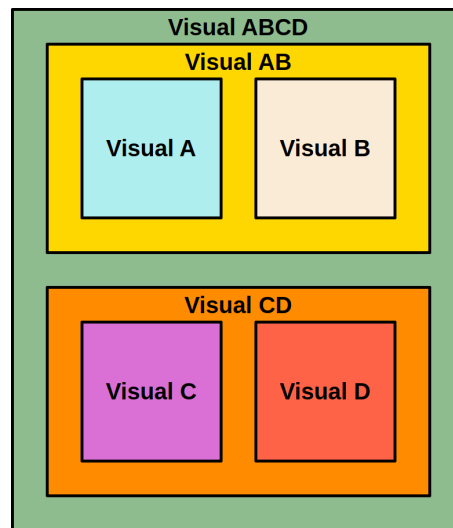


Figure 4.1: Composition of the LayerIt visualisation system. Each coloured box represents a distinct visual element that composes a higher-level visualisation.

This “layers within layers” design is applied at three levels of the project: the **visual level**, where visualisations combine aligned data, audio, and musical elements; the **SVG level**, where LayerIt organizes the XML structure of the output; and the **code level**, where the OO architecture organizes the corresponding program components. Together, these levels describe how individual elements are combined into composite visualisations.

#### 4.6.1 Visual Level

At the visual level, layers are output elements aligned to a shared timeline. They can be arranged vertically in a composite view while retaining the temporal correspondence between the score, audio-derived representations, annotations, and BT output.

#### 4.6.2 SVG Level

At the SVG level, LayerIt organizes the XML structure of the output so that individual and combined layers remain identifiable. The following listing shows a simplified representation of the XML structure produced for a composed SVG file:

Listing 4.2: Example structure of an SVG file

```
1 <svg>
2 |   <Top Visualisation>
3 | |   <Layers>
4 | | |   <image spectrogram.png>
5 | | |   <BeatThis Output>
6 | | |   <Onsets>
7 | |   <Score>
8 | |   <timeAxis>
9 |   <Bottom Visualisation>
10 | |   <Layers>
11 | | |   <Waveform>
12 | | |   <Beat Probability>
13 | | |   <Downbeat Probability>
14 </svg>
```

This XML organisation allows the SVG to be inspected and modified directly through its markup. It also preserves an identifiable structure for code-based processing while keeping the SVG readable. The nesting of elements at the XML level mirrors the visual composition described above.

### 4.6.3 Code Level

At the code level, LayerIt’s OO architecture organizes each visualisation component as a self-contained module with a consistent interface. The same composition principle used at the visual and SVG levels is therefore represented in the code structure. The following snippets illustrate the base abstraction, the `Visualizer` composition API, and a short end-to-end workflow.

Listing 4.3 shows an illustrative `Layer` base-class interface. The omitted imports are not relevant to the interface.

Listing 4.3: Layer interface (concise)

```

1 class Layer(ABC):
2     def __init__(self, name: str = "Layer"):
3         self._data = None
4         self.name = name
5
6     @abstractmethod
7     def load_data(self, **kwargs) -> bool:
8         pass
9
10    @abstractmethod
11    def draw(self, ax: Axes, shared_data: Dict[str, Any]) \
12        -> Tuple[List, List]:
13        pass
14
15    def to_svg_group(self, shared_data: Dict[str, Any]) \
16        -> Optional[str]:
17        '''Optional SVG export for subclasses'''
18    return None

```

In `LayerIt`, a `Visualizer` instance orchestrates the composition of multiple layers through a panel-based workflow. Layers are grouped into panels using `add_panel()`, which allows multiple layers to be rendered together on the same axis. The `Visualizer` maintains a shared data dictionary that all layers access during rendering via `load_all_layers()` and `draw()`. The `compose()` method then renders all panels and combines them with the warped score into a final multi-panel SVG. The next listing (4.4) illustrates this composition workflow.

Listing 4.4: Visualizer Panel Composition

```

1 # Create a visualizer with shared data paths
2 visualizer = Visualizer(audio="performance.wav",
3                         score="score.svg",
4                         maps="alignment.maps.json",
5                         beats="beats.npz")
6
7 # Add a panel (group of layers on same axis)
8 visualizer.add_panel(
9     BeatLogits(color='red'),
10    DownbeatLogits(color='blue'),
11    show_axes=True,
12    height_scale=0.5
13 )
14
15 # Compose all panels into final figure
16 visualizer.compose("output.svg", score_position=0)

```

In the composition workflow, the `Visualizer.compose()` method orchestrates the rendering of all panels and their combination with the warped score. Internally, each panel is rendered to SVG through `turn_to_SVG()`, which preserves each layer as a named SVG group. These individual panel SVGs are then combined with the warped score SVG using `create_final_SVG()`, which stacks them vertically at defined y-offsets while preserving the temporal alignment through the common timeline. The following listing shows this internal composition workflow.

Listing 4.5: SVG Composition Workflow

```
1 # Internal workflow (called by compose())
2
3 # For each panel: render layers, export to SVG
4 panel_viz = Visualizer(audio=audio, score=score,
5                       maps=maps, beats=beats)
6 panel_viz.add_panel(MelSpec(), BeatLogits())
7 panel_viz.load_all_layers(audio_path=audio,
8                          beat_file=beats)
9 fig, ax = panel_viz.draw()
10 panel_svg = panel_viz.turn_to_SVG(
11     filename="panel.svg",
12     svg_warped_score=score)
13
14 # Stack panels with score using create_final_SVG()
15 composer = Visualizer(audio=audio, score=score,
16                      maps=maps, beats=beats)
17 composer.create_final_SVG(
18     svg_layers=[(panel_svg, 50),
19                (score_svg, 300)],
20     output_file="final.svg")
```

The example shows that higher-level SVG outputs can be positioned or stacked vertically, while lower-level Matplotlib layers can be composed into them. This compositional behaviour implements the “layers within layers” design of LayerIt.

LayerIt supports a workflow from matplotlib-based visualisation generation to SVG or PNG composition. The final composition stage can include externally produced graphics, allowing code-generated visualisations to be combined with images created outside the codebase.

This hybrid design supports multiple workflows, including Python-based prototyping and the use of externally generated images. Custom layer implementations and external image inputs can both be incorporated into the same composite visualisation.

These code-level choices provide the compositional mechanism evaluated in Chapter 5. The case study demonstrates that existing layers can be modified, exchanged, and regenerated through the common interface. The addition of entirely new layer types remains a direction for future work rather than a quantified result of this thesis.

## 4.7 Summary

This chapter closes the methodological and implementation stages of the thesis by connecting the development process described in Chapter 3 with the framework described in this chapter. The iterative path taken through the five prototypes clarified the scope of the work, showing that the most meaningful contribution was not a new annotation system, but a reusable visualisation framework grounded in time-aligned music data.

LayerIt was then shaped to answer that need: a Python OO framework for composing score-aligned MIR visualisations through layered SVG-based output. Its design emphasizes reuse, composability, and visual consistency, while relying on ScoreWarp for the alignment backbone and on established MIR libraries for the generation of the underlying audio and data layers. Taken together, the two chapters show how the project moved from exploratory prototyping to a focused implementation that can now be assessed against the criteria established in the methodology chapter.

## Chapter 5

# Validation

This chapter evaluates LayerIt against the criteria defined in [chapter 3](#): coverage, reproducibility, composability, and fidelity and limits. Since a controlled user study was outside the scope of this work, the evaluation is qualitative. It combines a capability demonstration based on figures generated with LayerIt, a contextual comparison with existing workflows, and an explicit discussion of the evidence that remains to be collected.

### 5.1 Validation Approach

The validation considers whether the framework can express the score-aligned, multi-modal figures that motivated the thesis, and whether its composition model supports their creation, modification, and regeneration once a score-performance alignment is available. The objective is not to claim that LayerIt replaces interactive annotation software, reduces annotation effort, or establishes the accuracy of an alignment algorithm. Instead, it assesses the framework as a reproducible means of composing visual material from modular layers.

Three of the four figures in this chapter are original outputs generated locally from the same case study: J.S. Bach’s fugue from BWV 856, performed by András Schiff. The inputs are the audio recording, its MEI score, a MAPS file, the ScoreWarp-generated SVG score, and BeatThis output. The remaining figure ([Figure 5.1](#)) uses a second case study, the opening bars of Debussy’s *Clair de Lune* performed by Maria João Pires, to demonstrate the framework’s audio-representation layers independently of BT data. The figures are therefore not reproductions of published images. Published work is referenced only to motivate the figure types and the workflow gap addressed by the framework.

The evaluation distinguishes one-time preparation from per-figure work. Producing a score-aligned figure requires preparing the MEI score, MAPS annotations, warped score, and, where relevant, algorithmic output. These are substantial prerequisites shared by all LayerIt figures. Once they are available, the scripts supplied with this thesis define and compose the desired layers, and can be re-run when the input data or visual parameters change.

## 5.2 Reproducing Composite Figure Types (RQ1)

RQ1 asks how heterogeneous MIR visualisations can be combined in a common score-aligned representation. The four examples in this section address this question by composing symbolic notation, audio-derived representations, onset annotations, and BT output around a warped score timeline. They cover both dense, multi-layer views and a time-based display.

### 5.2.1 Score, audio, and chroma on a shared axis

The first example combines a time-warped score, a waveform, a mel spectrogram, and a chromagram for the opening six bars of Debussy's *Clair de Lune*, performed by Maria João Pires. Cyan dotted onset markers are repeated across the spectrogram and waveform, making their common performance-time reference visible. The warped score allows for the notation and the audio representations to be read against the same time axis allowing for a more contextualized and informed visualization.

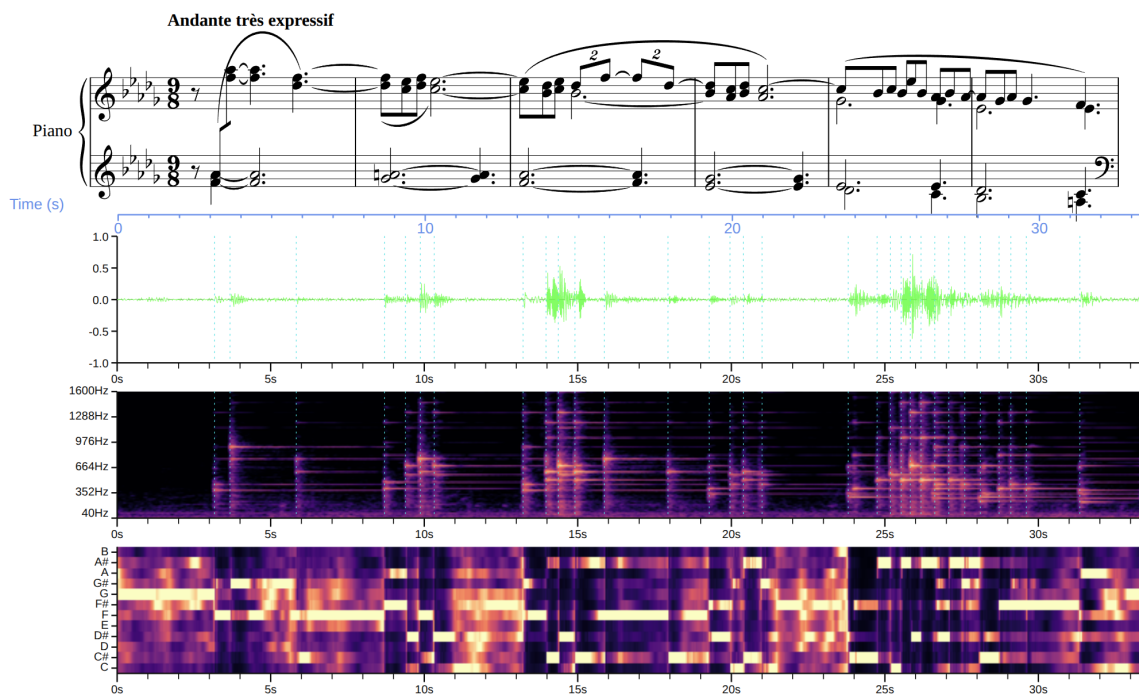


Figure 5.1: LayerIt composition for the opening six bars of Debussy's *Clair de Lune*, performed by Maria João Pires. From top to bottom: time-warped score, waveform, mel spectrogram, and chromagram. Cyan dotted lines mark score aligned onsets.

### 5.2.2 Score-aligned BT analysis

The second example places a time-warped score, a spectrogram, BeatThis beat estimates and onset markers in a single composition. The score provides musical context directly beneath the time-based data. This is valuable when interpreting a BT result because the reader can relate an



The figure type is motivated by the visualisation in Sá Pinto’s BT evaluation work [19] where musical context and time-based analysis are complementary but not physically aligned. LayerIt does not change the evaluation procedure proposed in that work; it provides a way to present the relevant score, spectral, and beat information on a shared axis.

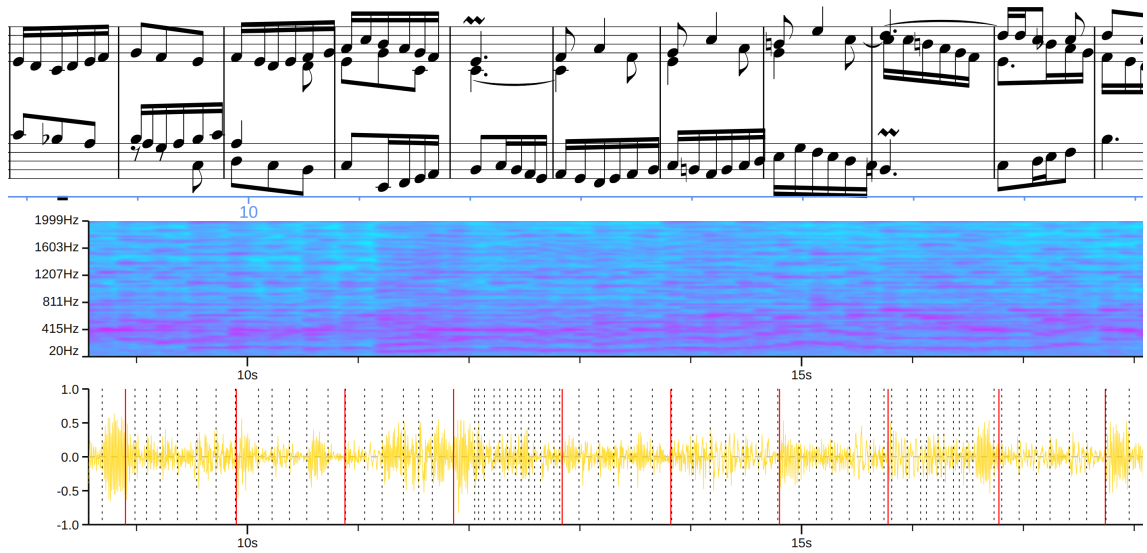


Figure 5.2: LayerIt score-aligned BT display for BWV 856, performed by András Schiff. From top to bottom: time-warped score, mel spectrogram, and normalized waveform with BeatThis beat and downbeat markers. All panels share the performance-time axis.

### 5.2.3 Logits and position displays

BT analysis benefits from retaining both the discrete output of an algorithm and its frame-wise activation/logits functions. Figure 5.3 presents two such views. In the upper view, detected beat positions are displayed together with the corresponding activation curve. In the lower view we see the same data this time for downbeat estimates and their respective logits. In both cases, the score, the logits curves and beat/downbeat estimates, remain tied to the same horizontal axis.

This type of visualization can help BT researchers better understand and relay how their algorithms perform. In this case we can visualize how the estimates for beat and downbeat differ.

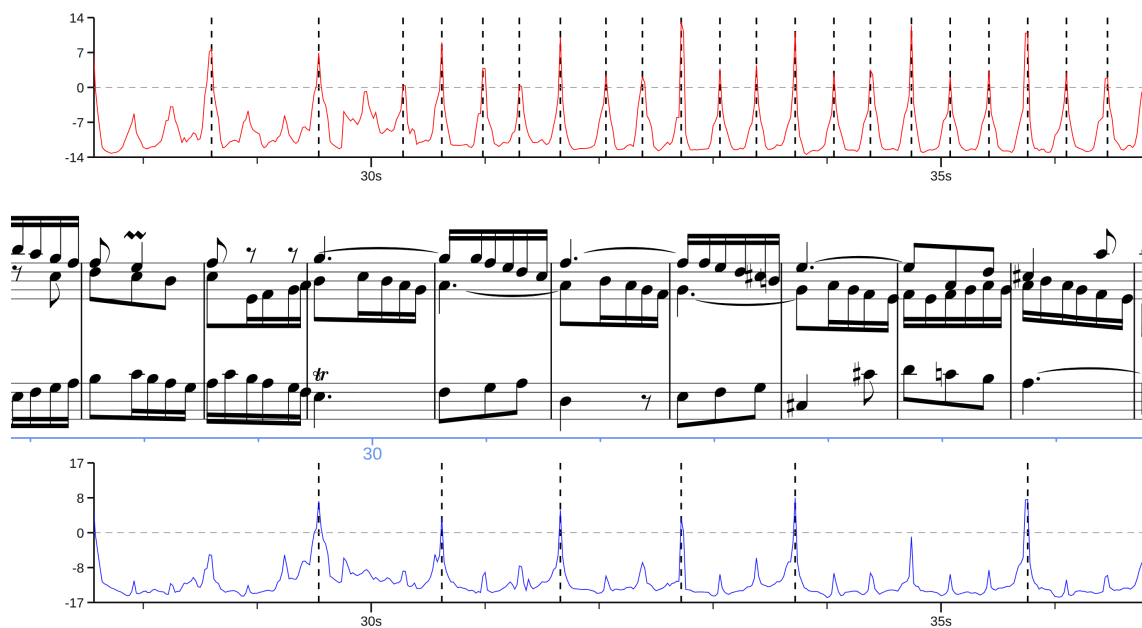


Figure 5.3: LayerIt activation-and-position display for BWV 856. The upper panel shows BeatThis beat logits and selected beat estimates; the time-warped score occupies the middle panel; and the lower panel shows downbeat logits and estimated downbeat positions.

#### 5.2.4 Layered time-based display

The final example demonstrates the lightweight end of the composition model. It overlays a normalized waveform with BeatThis beat estimated positions, then places the warped score above it. Although it contains fewer layers than the previous examples, it uses the same composition process and preserves the same time relationship. This supports rapid inspection of how detected beats relate to both acoustic events and notated musical material.

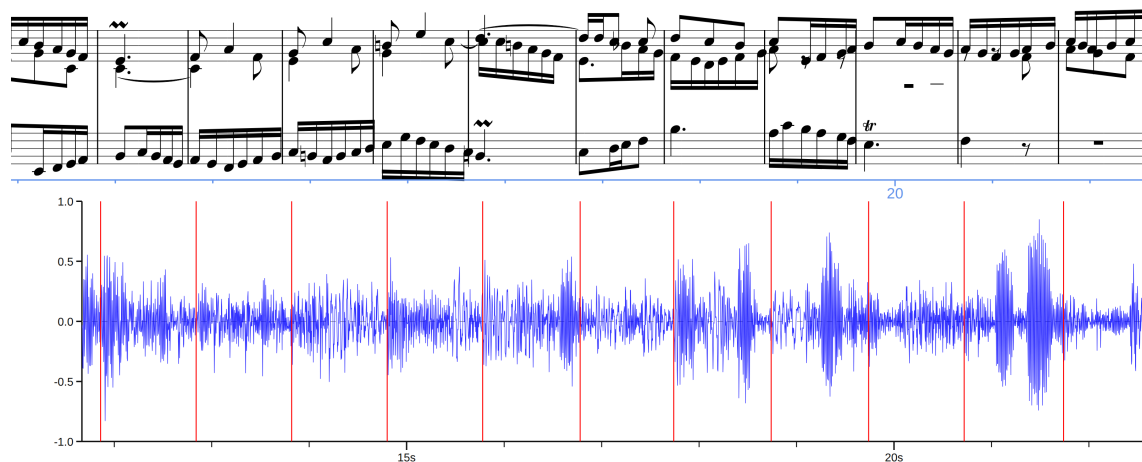


Figure 5.4: LayerIt waveform display an excerpt for BWV 856 by András Schiff, with BeatThis beat positions aligned to the time-warped score.

Table 5.1: Composite figure types reproduced with LayerIt.

| Figure type                     | Motivating context                    | LayerIt layers                                                       | Composition result                                                     |
|---------------------------------|---------------------------------------|----------------------------------------------------------------------|------------------------------------------------------------------------|
| Score, audio, and chroma        | Multimodal music representations [14] | Warped score, waveform, onset markers, mel spectrogram, chroma-gram  | Multiple audio representations aligned to the score                    |
| Score-aligned BT analysis       | BT evaluation [19]                    | Warped score, mel spectrogram, waveform, beat and downbeat positions | Musical context, acoustic evidence, and algorithmic output in one view |
| Activation and position display | Qualitative inspection of BT output   | Beat and downbeat logits, selected positions, warped score           | Continuous activations and discrete output compared on one axis        |
| Layered time-based display      | Compact BT inspection                 | Warped score, waveform, beat positions                               | A minimal score-aligned analytical display                             |

Taken together, these examples answer RQ1 at the level of framework capability: the layer model can compose heterogeneous visualisations into a common score-aligned representation. The evidence is a set of independently re-runnable scripts rather than a manually assembled figure. The examples do not establish universal visual design rules; they show that the required figure types are expressible by the framework.

### 5.3 Workflow Capabilities for Annotation and Evaluation (RQ2, RQ3)

This section positions LayerIt alongside Sonic Visualiser, Piano Precision, and manual SVG editing in relation to the workflows that motivated the project. It concerns the production and inspection of score-aligned figures, rather than a complete feature comparison of the other tools. Sonic Visualiser and Piano Precision remain purpose-built interactive applications, whereas LayerIt is a scriptable framework for generating static, structured SVG images.

#### 5.3.1 Evaluation workflow

For BT evaluation, the principal advantage of LayerIt is that the algorithm’s discrete positions and continuous activation functions can be read against the same time-warped score. [Figure 5.3](#) illustrates this combination: the reader can inspect peaks in the beat and downbeat activations, the positions selected by the algorithm, and the corresponding notation without translating between separate time references.

This supports qualitative diagnosis. For example, a selected beat can be compared with a nearby activation peak and with the score event at the same horizontal position. Conversely, an activation peak that does not result in a selected position can be identified and interpreted in musical context. Sonic Visualiser supports rich audio inspection, but the workflow considered here

requires an additional score-aligned composition. Piano Precision provides score-performance links for navigation and analysis [10]; however, its link-based score view does not provide the continuously time-warped score layer required by these static composite figures.

### 5.3.2 Annotation workflow

LayerIt is not an interactive annotation environment and does not itself let an annotator place or correct beat times. Its contribution to annotation is instead a visual reference for an upstream annotation workflow. An annotator can inspect the alignment between the spectrogram or waveform, onset markers, candidate beat positions, and the time-warped score; corrections are then made in the relevant annotation or MAPS source and the figure is regenerated.

In this workflow, the score is physically positioned on the same axis as the data to be inspected. This differs from a link-based score display, where correspondence is expressed through navigation or connections between separately laid out views. For tasks that require deciding whether a candidate beat coincides with a notated event, the shared-axis view provides direct visual context. It should nevertheless be understood as a diagnostic and communication aid, not as evidence that LayerIt improves annotation speed or accuracy; such a claim would require an empirical user study.

Table 5.2: Workflow characteristics relevant to the score-aligned visualisation task considered in this thesis.

| Dimension                                         | Sonic Visualiser                    | Piano Precision                 | Manual editing                               | SVG | LayerIt                                 |
|---------------------------------------------------|-------------------------------------|---------------------------------|----------------------------------------------|-----|-----------------------------------------|
| Shared score time axis in the workflow considered | Not provided                        | Link-based score view           | Possible through manual placement            |     | Provided through ScoreWarp              |
| Arbitrary algorithmic curves                      | Limited by available layers/plugins | Outside the workflow considered | Possible through manual construction         |     | Provided through Layer subclasses       |
| Scriptable figure generation                      | Not assessed                        | Not possible                    | Depends on a separately maintained procedure |     | Provided through the composition script |
| Regeneration after data change                    | Manual reconfiguration              | Manual view recreation          | Repeat manual editing                        |     | Re-run with updated inputs              |
| Adding a visualisation type                       | Plugin development                  | Plugin development              | Manual construction                          |     | Implement or configure a Layer          |

No controlled count of manual operations or wall-clock time was recorded for the workflows considered here. Consequently, this thesis does not report numerical claims about time saved by LayerIt. The evidence for RQ2 is functional rather than comparative: once the inputs are prepared, the supplied scripts generate the demonstrated figures without manual image alignment, and the same scripts can be re-run after changes to the data or visual parameters. A future study should measure one-time setup and per-figure effort separately.

## 5.4 Modular Composition Evidence (RQ2)

The architecture described in [chapter 4](#) defines a common `Layer` interface and composes layers through `Visualizer`. The examples in this chapter exercise that interface with several independent layer types including: spectrogram, chromagram, waveform, onsets layer, BT output layers, beat/downbeat activation curves. Each is added through the same `add_panel()` operation and rendered through the common `compose()` workflow.

The scripts provide practical evidence of composability: [Figure 5.1](#) combines four different representation types, while [Figure 5.3](#) combines separate beat and downbeat activation and event layers around the same score. These substitutions require no change to the composition mechanism, which is consistent with the modular claim of Chapter 4.

The current evidence supports the claim that existing layer types can be composed and exchanged through a common interface. It does not quantify the implementation cost of adding a completely new visualisation type, because no new layer was introduced solely for this evaluation with a measured line count and a documented change to the framework core. Such a probe should be added in future work by implementing one new layer and documenting whether any change outside that layer is required.

## 5.5 Limitations and Difficult Cases (RQ3)

The examples in this chapter depend on a reasonably faithful correspondence between the performance and the score. `LayerIt` composes and scales visual layers against the `ScoreWarp` timeline, so the fidelity of the composite visualisation is bounded by the quality and density of the underlying alignment data. This is an important distinction: a misleading score position caused by an inaccurate MAPS alignment is not a failure of the layer-composition mechanism, but the final figure nevertheless inherits that limitation.

### 5.5.1 Expressive performances

Highly expressive performances, including substantial rubato, skipped material, repetitions, or departures from the notated score, can make the relationship between score position and performance time difficult to represent with a simple warped timeline. Structural discontinuities such as repeats and jumps are a recognised challenge for score-performance synchronization [7, 21]. In such cases, a score element may be placed at a time that does not adequately describe the local musical event, and the misalignment then propagates visually to every layer that uses the same reference.

No dedicated expressive-performance stress test was completed for this thesis. The present evidence therefore establishes this as a boundary of the approach rather than measuring a specific degradation pattern. A suitable follow-up evaluation would compare dense and manually verified MAPS annotations for an expressive performance with a less expressive reference performance, then report where the warped score remains informative and where it ceases to be reliable.

### 5.5.2 Sparse onset annotation

Sparse MAPS annotations present a related limitation. ScoreWarp can interpolate between available anchors, reducing the human effort required to prepare a score-performance alignment. However, the temporal position between widely separated anchors is then an interpolation rather than a directly supported observation. The resulting score placement may be sufficient for an overview, but it is less appropriate for detailed beat-level inspection in passages with tempo variation.

Figure 5.5 reports this comparison for an excerpt of BWV 865 performed by András Schiff. The upper pair of panels uses a sparse MAPS file that retains the first and last onsets and every fiftieth onset between them. The lower pair uses the complete onset file. In both cases, the red vertical lines show the BeatThis beat estimates, while the black dotted lines show the onsets retained by the corresponding MAPS file.

The sparse alignment preserves the broad placement of the excerpt and remains most useful near the retained anchors. Between anchors, however, the score position is determined by interpolation rather than by a directly observed onset, so local spacing becomes less reliable. The dense alignment supplies onset references throughout the passage and keeps the score and waveform correspondence more stable, including around passages with several closely spaced notes. This comparison supports the expected trade-off: sparse anchors reduce preparation effort but provide weaker evidence for detailed beat-level inspection, whereas dense anchors support more reliable local placement at the cost of additional annotation data.

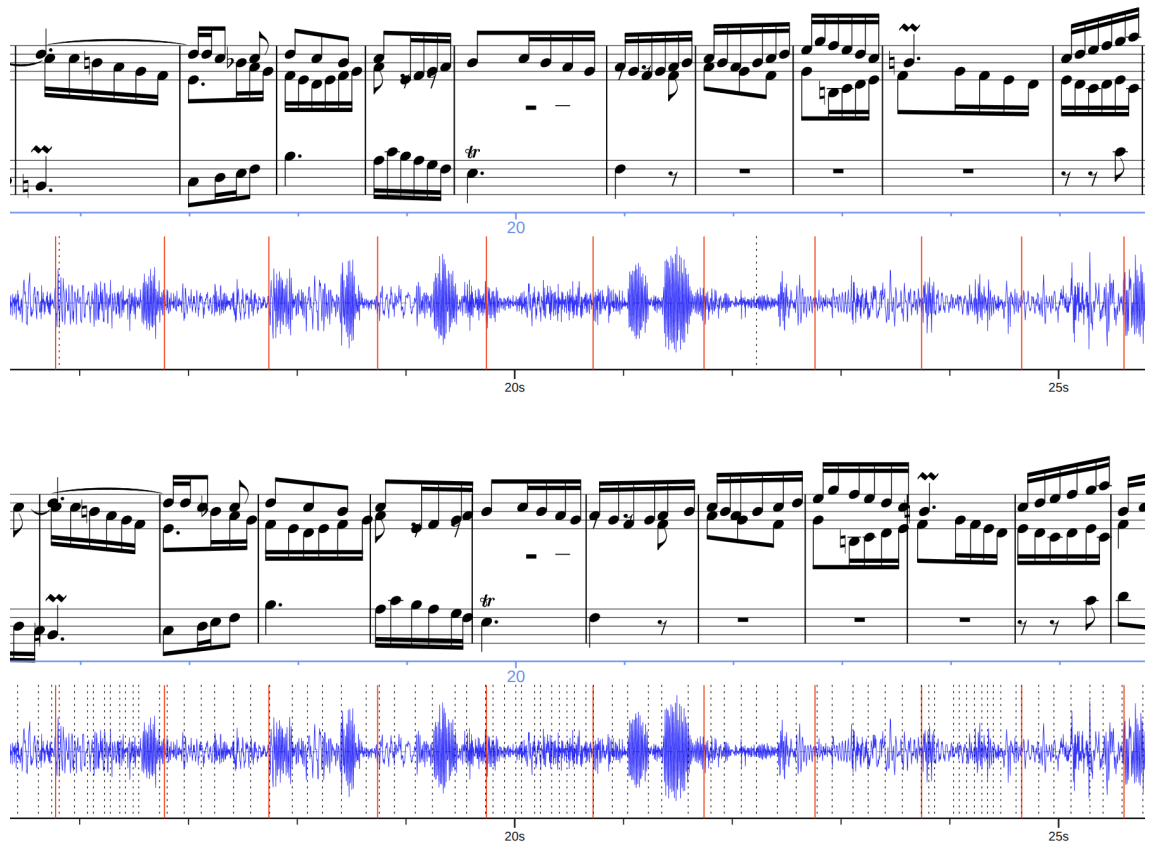


Figure 5.5: Comparison of sparse and dense score–performance alignment for an excerpt of BWV 865 performed by András Schiff. From top to bottom: score warped using sparse onsets, waveform with BeatThis beat estimates in red and retained sparse onsets as black dotted lines, score warped using all onsets, and the same waveform with the complete onset set. The sparse MAPS file retains the first and last onsets and every fiftieth onset between them.

### 5.5.3 Manual editing of the final SVG

Manual editing of a final SVG is not required for visual customisation in LayerIt. The framework exposes customisation through its layer configuration and composition code, allowing users to adjust existing layers and incorporate the visual elements required for a particular analysis while retaining a re-runnable generation script. It can also include externally prepared PNG and SVG assets, allowing a composite visualisation to combine LayerIt-generated layers with other image-based material.

When an existing layer does not provide the required behaviour, the OO architecture supports modifying or extending the codebase by implementing a new `Layer` or adapting an existing one. This keeps the visual logic within the framework and preserves traceability from the source data and configuration to the resulting figure. Direct hand-editing of the exported SVG remains possible as a fallback, but it should be avoided when the adjustment can instead be represented in code, since unrecorded manual changes weaken reproducibility. If such editing is necessary, the

modified SVG and the reason for the modification should be retained alongside the generation script.

## 5.6 Summary

This chapter provides a qualitative answer to the three research questions. For RQ1, the four figures demonstrate that LayerIt can compose symbolic score material, audio representations, onset annotations, beat and downbeat activations, and BT output into common score-aligned views. For RQ2, the generated examples and composition workflow show that existing layers can be created, modified, exchanged, and regenerated through a common scriptable interface.

For RQ3, the examples show how a shared-axis visualisation can support qualitative inspection of BT results and can serve as a diagnostic reference for annotation. LayerIt is not itself an interactive annotation tool, and no user-study claim is made. Its usefulness is also conditioned by the quality of the score-performance alignment: expressive performances and sparse annotation anchors remain important limitations that require dedicated empirical evaluation.



## Chapter 6

# Conclusions and Future Work

### 6.1 Achievements

This thesis set out to address the lack of a reusable framework for composing score-aligned, multi-modal MIR visualisations and to demonstrate how such a framework can provide qualitative context for beat annotation and BT evaluation workflows. Revisited against the objectives and research questions set out in [chapter 1](#), the work can be summarised as follows.

- **Objective 1** was met through the development of LayerIt, a Python OO framework in which score, audio, and algorithmic data are treated as independent, composable layers. [chapter 4](#) showed that this “layers within layers” philosophy holds consistently at the visual, SVG, and code levels, giving the framework a coherent architecture rather than a single-purpose script.
- **Objective 2** was met through the generation and regeneration of four distinct composite figure types. These include a *Clair de Lune* view combining a warped score, waveform, mel spectrogram, and chromagram; BWV 856 views combining score, spectrogram, waveform, beat and downbeat events, and activation curves; and a compact score-and-waveform display. The same composition mechanism creates the figures, and the demonstrated layers can be modified or exchanged through the common Layer and Visualizer interfaces. This provides functional evidence for RQ2 without making comparative claims about development time or user effort.
- **Objective 3** was to assess whether LayerIt could support the interpretation of BT and BA data. The BWV 856 and *Clair de Lune* case studies show that placing the score, audio representations, tracker activations, and estimated events on one performance-time axis gives these data a shared musical context. This makes it possible to inspect disagreements between the tracker and the notated metre, and to relate an estimated event to both the recording and the corresponding score passage. The result is a visual diagnostic aid rather than an annotation tool or a beat-tracking evaluation by itself: its usefulness depends on the quality of

the score-performance alignment, especially for expressive performances and sparse onset annotations.

This thesis produced concrete, results namely: LayerIt as an open-source Python framework with example scripts for demonstration purposes and a Late-Breaking Demo submitted to ISMIR 2026. Just as importantly, the iterative process described in [chapter 3](#) is itself an achievement of the work: by building and discarding five successive prototypes, the thesis identified that beat-annotation and BT visualisation are, in practice, two separate problems, and it deliberately narrowed its scope to the one that offered the clearest and most feasible contribution. That decision allowed LayerIt to be evaluated against criteria appropriate to a visualisation framework rather than an interactive GUI.

## 6.2 Future Work

The limitations identified in [chapter 5](#) point directly to the most pressing follow-up work. First, the current evidence demonstrates the composition of existing layers but does not quantify the effort required to introduce a completely new layer; a controlled extensibility study could implement a new `Layer` subclass and document the changes it requires. Second, the fidelity of LayerIt’s visualisations is bounded by the quality of the underlying score-performance alignment; dedicated experiments comparing dense and sparse MAPS annotations, and stress-testing the framework against expressive performances with substantial rubato or structural deviations from the score, would clarify precisely where the time-warped representation remains reliable.

Beyond validating what already exists, several directions could extend LayerIt’s scope. The framework currently integrates a single alignment method (ScoreWarp) and a single BT algorithm (BeatThis). A synchronisation toolkit such as Sync Toolbox could provide additional upstream score-performance alignments [15], while support for an interchange representation such as the match file format could make these inputs more portable [6]. Supporting these and other alignment and MIR algorithms as further `Layer` implementations would test the framework under more demanding conditions and grow LayerIt into a more general-purpose visualisation toolkit. Validating the framework against a broader range of instrumentations, genres, and performance styles would strengthen the generality of its conclusions. Finally, having deliberately set aside the interaction problem in [chapter 3](#), a natural long-term direction is to revisit it: the SVG output produced by LayerIt already preserves the addressable structure inherited from Verovio and MEI, which could be leveraged to build an interactive, browser-based annotation layer on top of the visualisations this thesis already knows how to generate. Pursuing this direction would reconnect the visualisation contribution made here with the beat-annotation and BT-evaluation tools that originally motivated this thesis.

The output format also suggests two technical extensions. First, accepting richer alignment inputs with provenance and domain information would make it easier to distinguish directly observed anchors from interpolated score positions. Second, dense time-frequency and pitch fields

are currently embedded as raster images inside otherwise structured SVG output. This preserves the visual result but limits post-export restyling compared with the vector score and event layers. Future work should investigate metadata-preserving field layers or controlled vector representations where the resulting file size and rendering cost remain practical.

### 6.3 Final Remarks

This thesis began with the observation that producing time-aligned, multi-modal visualisations can require the manual combination of separately generated score, audio, and algorithmic representations. Through an iterative development process and careful architectural design, we developed LayerIt, a Python-based OO tool for composing these representations as structured, score-aligned visualisations.

LayerIt's value lies in its demonstrated ability to compose visual layers into re-runnable, score-aligned figures for BT evaluation and beat annotation inspection. The score remains identifiable through the MEI and Verovio-derived SVG structure, while added components remain separate and restylable where they are represented as vector elements. By grounding the system in established conventions, including OO design patterns, common MIR libraries, and standardised formats, the framework provides a basis for future extension and integration.

As MIR research continues to evolve through advances in machine learning, increasing dataset complexity, and growing interdisciplinary collaboration, the need for flexible visualisation tools will continue to grow. LayerIt provides a focused foundation for future work on composable score-aligned visualisations.

# References

- [1] *An Introduction to MEI*. URL: <https://music-encoding.org/about/>.
- [2] Chris Cannam, Christian Landone, and Mark Sandler. “Sonic Visualiser: An Open Source Application for Viewing, Analysing, and Annotating Music Audio Files”. In: *Proceedings of the 18th ACM International Conference on Multimedia (MM ’10)*. Florence, Italy: ACM Press, 2010, pp. 1467–1468. DOI: [10.1145/1873951.1874248](https://doi.org/10.1145/1873951.1874248).
- [3] J.J. Carabias et al. “An Audio Score Alignment Framework Using Spectral Factorization And Dynamic Time Warping”. In: (Oct. 2015).
- [4] Zhiyao Duan and Bryan Pardo. “Soundprism: An Online System for Score-Informed Source Separation of Music Audio”. In: *IEEE Journal of Selected Topics in Signal Processing* 5.6 (Oct. 2011), pp. 1205–1215. ISSN: 1932-4553, 1941-0484. DOI: [10.1109/JSTSP.2011.2159701](https://doi.org/10.1109/JSTSP.2011.2159701). URL: <http://ieeexplore.ieee.org/document/5887382/> (visited on 06/07/2026).
- [5] Francesco Foscarin, Jan Schlüter, and Gerhard Widmer. “Beat This! Accurate Beat Tracking without DBN Postprocessing”. In: *Proceedings of the 25th International Society for Music Information Retrieval Conference (ISMIR)*. San Francisco, CA, USA, 2024. DOI: [10.48550/arXiv.2407.21658](https://doi.org/10.48550/arXiv.2407.21658). (Visited on 02/04/2025).
- [6] Francesco Foscarin et al. “The Match File Format: Encoding Alignments between Scores and Performances”. In: *Proceedings of the Music Encoding Conference*. Halifax, NS, Canada, 2022, pp. 52–60. DOI: [10.48550/arXiv.2206.01104](https://doi.org/10.48550/arXiv.2206.01104). (Visited on 08/19/2026).
- [7] Christian Fremerey, Meinard Müller, and Michael Clausen. “Handling Repeats and Jumps in Score-Performance Synchronization”. In: *Proceedings of the 11th International Society for Music Information Retrieval Conference*. 2010, pp. 243–248.
- [8] Werner Goebl et al. “Let’s Do the ScoreWarp Again! Shifting Notes to Performance Timelines”. In: *Proceedings of the Music Encoding Conference*. London, United Kingdom, 2025. DOI: [10.17613/gqd3b-w7c28](https://doi.org/10.17613/gqd3b-w7c28). (Visited on 06/05/2026).
- [9] Johannes Hentschel et al. “Time to Align! Modelling Musical Timelines for Music Information Retrieval and Digital Musicology”. In: *Transactions of the International Society for Music Information Retrieval* 9.1 (2026), pp. 384–404. DOI: [10.5334/tismir.296](https://doi.org/10.5334/tismir.296). (Visited on 08/19/2026).
- [10] Yucong Jiang, David M. Weigl, and Werner Goebl. “Piano Precision: Advancing Music Performance Analysis by Integrating User-Correctable Audio-to-Score Alignment”. In: *Proceedings of the 22nd Sound and Music Computing Conference (SMC)*. Graz, Austria, July 2025, pp. 160–167. DOI: [10.5281/zenodo.15838699](https://doi.org/10.5281/zenodo.15838699). URL: <https://doi.org/10.5281/zenodo.15838699>.

- [11] Jongmin Jung et al. “Unified Cross-modal Translation of Score Images, Symbolic Music, and Performance Audio”. In: *IEEE Transactions on Audio, Speech and Language Processing* 34 (2026), pp. 1876–1891. arXiv: 2505.12863 [cs.SD]. URL: <http://arxiv.org/abs/2505.12863> (visited on 06/02/2026).
- [12] Zulfidin Khodzhaev. “A practical guide to spectrogram analysis for audio signal processing”. In: *arXiv preprint arXiv:2403.09321* (2024).
- [13] Brian McFee et al. *Librosa: Audio and Music Signal Analysis in Python*. Austin, Texas, USA, 2015.
- [14] Meinard Müller. *Fundamentals of music processing: Audio, analysis, algorithms, applications*. Vol. 5. Springer, 2015.
- [15] Meinard Müller et al. “Sync Toolbox: A Python Package for Efficient, Robust, and Accurate Music Synchronization”. In: *Journal of Open Source Software* 6.64 (2021), p. 3434. DOI: 10.21105/joss.03434. (Visited on 08/19/2026).
- [16] Eita Nakamura, Kazuyoshi Yoshii, and Haruhiro Katayose. “Performance Error Detection and Post-Processing for Fast and Accurate Symbolic Music Alignment.” In: (2017), pp. 347–353.
- [17] Silvan David Peter et al. “Automatic Note-Level Score-to-Performance Alignments in the ASAP Dataset”. In: *Transactions of the International Society for Music Information Retrieval* 6.1 (June 26, 2023), pp. 27–42. ISSN: 2514-3298. DOI: 10.5334/tismir.149. URL: <http://transactions.ismir.net/articles/10.5334/tismir.149/> (visited on 06/07/2026).
- [18] Répertoire International des Sources Musicales. *Verovio Reference Book*. 2021. DOI: 10.5448/7EM6-MY23. URL: <https://book.verovio.org/> (visited on 06/06/2026). Pre-published.
- [19] António Sá Pinto, Inês Domingues, and Matthew E. P. Davies. “Shift If You Can: Counting and Visualising Correction Operations for Beat Tracking Evaluation”. In: *Late-Breaking Demo Session of the 21st International Society for Music Information Retrieval Conference*. 2020. DOI: 10.48550/arXiv.2011.01637.
- [20] The Matplotlib Development Team. *Matplotlib: Visualization with Python*. Version v3.10.9. Zenodo, Apr. 24, 2026. DOI: 10.5281/ZENODO.19716234. URL: <https://zenodo.org/doi/10.5281/zenodo.19716234> (visited on 06/22/2026).
- [21] Verena Thomas et al. “Linking Sheet Music and Audio – Challenges and New Approaches”. In: *Multimodal Music Processing*. Vol. 3. Dagstuhl Follow-Ups. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2012, pp. 1–22. DOI: 10.4230/DFU.Vol3.11041.1. URL: <https://doi.org/10.4230/DFU.Vol3.11041.1>.